

Estudiando Vercel GitHub y Componentes

Como Trabajan Juntos en Practica
y el Rol de Claude 4.7

Para Desarrolladores y Arquitectos

SaaS Factory V4
Para: Jose - Developer y Consultor
Mayo 26, 2026

GitHub: El Guardi an delCodigo

GitHub es el lugar donde vive el codigo. Es como un almacen seguro para tus archivos, pero con superpoderes.

LA HISTORIA: Por Que Existe GitHub

Antes de GitHub (antes de 2008):

- * Si tu PC se rompia, perdias TODO tu trabajo
- * No habia forma segura de colaborar con otros
- * Los cambios se guardaban en copias desorganizadas

GitHub resolvió este problema:

- * Guarda tu codigo en la nube (seguro)
- * Historial completo de cambios (puedes revertir)
- * Permite colaboracion de equipos (todos en lo mismo)
- * Registro de quien cambio que (transparencia)

LA MISION DE GITHUB

Ser la plataforma donde los desarrolladores colaboren sin perder codigo. Hacer el desarrollo seguro, organizado y accesible.

QUE HACE GITHUB

- * Almacena tu codigo en servidores seguros
- * Registra cada cambio (commits)
- * Permite ramas (trabajar en paralelo)
- * Facilita colaboracion en equipo
- * Envia notificaciones a otros sistemas (webhooks)

Vercel: El Publicador Automatico

Vercel es la plataforma que publica tu codigo en internet. Es como un empleado que siempre esta listo para poner tu app en vivo.

LA HISTORIA: Por Que Existe Vercel

Antes de Vercel (antes de 2016):

- * Publicar una app era complicado (muchos pasos)
- * Necesitabas configurar servidores manualmente
- * Tomar horas o dias entre escribir codigo y verlo en vivo
- * Requeria conocimiento tecnico especifico

Vercel resolvio este problema:

- * Publica automaticamente cuando subes a GitHub
- * Compila tu codigo en segundos
- * Publica en servidores globales al instante
- * No necesitas saber de infraestructura
- * App en vivo en minutos, no horas

LA MISION DE VERCEL

Hacer que publicar codigo sea tan facil como escribirlo. Eliminar la friccion entre desarrollo y produccion.

QUE HACE VERCEL

- * Escucha cambios en GitHub (mediante webhooks)
- * Descarga el codigo actualizado
- * Ejecuta npm run build (compila tu app)
- * Publica en servidores globales
- * Da una URL publica donde estan los usuarios

Como GitHub y Vercel Se Conectan

GitHub y Vercel no funcionan solos. Se conectan para crear automatizacion:

El Flujo de Conexion

1. TU subes codigo a GitHub (git push)
2. GitHub DETECTA que hay nuevos cambios
3. GitHub AVISA a Vercel (via webhook - un mensaje automatico)
4. Vercel RECIBE el aviso y actua automaticamente
5. Vercel DESCARGA el codigo de GitHub
6. Vercel COMPILA el codigo (npm run build)
7. Vercel PUBLICA en sus servidores
8. TUS USUARIOS ven los cambios en vivo

Por Que es Importante Esta Conexion

No es magia - es automatizacion inteligente. GitHub y Vercel se comunican solos, eliminando pasos manuales. Resultado: menos errores, mas velocidad.

Un Dia Tipico de Desarrollo

9:00 AM - Trabajas en tu codigo local

- * Editas: `src/app/biblioteca/page.tsx`
- * Cambias colores de botones a blanco
- * Testeas en `http://localhost:3000`

10:30 AM - Estas satisfecho. Haces commit

- * `git commit -m "fix: Cambiar botones a blanco"`

10:31 AM - Pusheas a GitHub

- * `git push origin master`
- * GitHub recibe tu codigo

10:32 AM - GitHub notifica a Vercel (webhook)

- * Vercel detecta: "Hay cambios en master"

10:33 AM - Vercel comienza el build

- * `npm run build`
- * Next.js compila
- * TypeScript valida
- * Tailwind CSS procesa

10:35 AM - APP EN VIVO

- * `https://gestioninventario.vercel.app`
- * Los usuarios VEN tus cambios

Los 3 Pilares Trabajando Juntos

1. GITHUB: El Guardian delCodigo

GitHub es DONDE VIVE tu codigo:

- * Versiones anteriores (puedes revertir)
- * Historial completo (quien cambio que)
- * Punto de disparo (webhook para Vercel)

2. VERCEL: El Publicador

Vercel es QUIEN PUBLICA tu codigo:

- * Detecta cambios en GitHub (escucha webhooks)
- * Compila tu codigo (npm run build)
- * Publica en servidores globales

3. COMPONENTES: Lo Que Se Publica

Son LO QUE se construye y publica:

- * React: componentes interactivos
- * Next.js: enrutamiento y API
- * TypeScript: validacion de tipos

Que Tener en Cuenta

IMPORTANTE: Los Errores Mas Frecuentes

ERROR 1: "Funciona en localhost pero no en Vercel"

Causa: Versiones diferentes de librerias

Solucion: npm run build localmente primero

ERROR 2: "Cambio no aparece en produccion"

Causa: Cache del navegador o de Vercel

Solucion: Hard refresh (Cmd+Shift+R) o espera 5 min

ERROR 3: "Build fallido en Vercel"

Causa: Error de TypeScript o dependencia

Solucion: npm run build en tu PC, corrige, luego push

ERROR 4: "Commits messy y confusos"

Causa: Mensajes de commit poco claros

Solucion: git commit -m "feat: descripcion clara"

Como Claude 4.7 Ayuda en Este Flujo

Claude 4.7: Tu Companero de Desarrollo

1. ENTENDER EL FLUJO

Claude analiza: "Que quiere Jose?"

Entiende: GitHub -> Vercel -> App en vivo

2. IDENTIFICAR PROBLEMAS

Claude ve: "Botones no estan blancos en prod"

Diagnostica: "Es Vercel alias? Es cache?"

3. PROPONER SOLUCIONES

Claude sugiere: "Vamos a chequear el deploy"

O: "Hagamos un rebuild forzado"

4. AUTOMATIZAR TAREAS

Claude: Edita codigo, hace commits, pushea

Resultado: Cambios en vivo sin intervencion

5. DEBUGGEAR ERRORES

Claude: Lee logs de Vercel, identifica raiz

Explica: "El problema es X, solucion es Y"

Cambiar Color de Botones en gestioninventario

SITUACION: Botones en color naranja

OBJETIVO: Cambiarlos a blanco

PASO 1: Editar el codigo

Archivo: src/app/layout.tsx

Cambio: style={{ color: "white" }}

PASO 2: Testear localmente

Comando: npm run dev

Resultado: Botones blancos en localhost:3000

PASO 3: Hacer commit

git commit -m "fix: Cambiar botones a blanco"

PASO 4: Pushear

git push origin master

GitHub recibe

PASO 5: Vercel deploya automaticamente

npm run build -> Next.js compila -> Deploy

APP EN VIVO: <https://gestioninventario.vercel.app>

PASO 6: Verificar en produccion

Abres la URL -> Botones BLANCOS

Reglas de Oro Que Debes Seguir

REGLA 1: Siempre testea localmente primero

Antes de pushear a GitHub, verifica en localhost

REGLA 2: Commits claros y descriptivos

NO: "arreglar cosas"

SI: "fix: Cambiar color botones a blanco"

REGLA 3: Un cambio por commit

No mezcles 5 cambios en un commit

REGLA 4: Revisa los logs de Vercel

Si algo falla en produccion, los logs te dicen por que

REGLA 5: Usa TypeScript siempre

Detecta errores antes de que lleguen a produccion

Problemas Comunes y Soluciones

Problema 1: "Port 3000 ya esta en uso"

Solucion: npm run dev usa puerto automatico

Problema 2: "TypeScript error pero el codigo ve bien"

Solucion: npm run typecheck para ver el error completo

Problema 3: "Vercel dice "build failed" pero localhost funciona"

Solucion: Verifica npm run build localmente

Problema 4: "Cambios no aparecen en produccion"

Solucion: Hard refresh o revisa el deployment en Vercel

Problema 5: "Necesito revertir un cambio"

Solucion: git revert <commit-hash> o git reset

El Ecosistema Completo

TODO TRABAJA JUNTO:

1. TU escribes codigo

En tu PC con Next.js, React, TypeScript

2. TU haces git push

El codigo sube a GitHub

3. GITHUB notifica a Vercel

Mediante webhook automatico

4. VERCEL construye y publica

npm run build -> Publica en servidores globales

5. USUARIOS ven los cambios

En produccion, en vivo, sin delay

6. CLAUDE 4.7 te ayuda en todo

Entiende, diagnostica, automatiza, debuggea

NO ES MAGIA - ES ARQUITECTURA BIEN DISEÑADA